

Automated Online Exam Proctoring System Using Computer Vision and Hybrid ML Classifier

Zarin Tahia Hossain
Computer Science and Engineering
Brac University
Dhaka, Bangladesh
zarinhossain31@gmail.com

Protyasha Roy
Computer Science and Engineering
Brac University
Dhaka, Bangladesh
protyasharoy263@gmail.com

Rina Nasir
Computer Science and Engineering
Brac University
Dhaka, Bangladesh
rinanasir25@gmail.com

Sumaiya Nawsheen
Computer Science and Engineering
Brac University
Dhaka, Bangladesh
sumaiya.naw252@gmail.com

Muhammad Iqbal Hossain
Computer Science and Engineering
Brac University
Dhaka, Bangladesh
iqbal.hossain@bracu.ac.bd

Abstract—Importance of online education can be seen especially during the ongoing Covid-19 when going to schools or colleges is not possible. So validity of online exams should also be maintained with respect to traditional pen-paper examinations. However, absence of invigilator makes it easy for the examinees to cheat during the exam. Though there are already many systems for online proctoring, not all educational institutes can afford them as the systems are very expensive. In this paper, we have used eye gaze and head pose estimation as the main features to design our online proctoring system. Therefore, the purpose of this paper is to use these features to create an online proctoring system using computer vision and machine learning and stop cheating attempts in exams.

Keywords— Online Proctoring; Machine Learning; MLP; XGBoost; Hybrid Classifier; Cheating Prediction

I. INTRODUCTION

To maintain the education system in crucial time like Covid-19, people came up with the solution which is online education. Yet, some are taking advantage of this situation completely to commit unfair means. Cheating in online examinations. It is high time, institutions and authorities take this problem seriously as it is one of the greatest reasons why online learning does not get similar value to traditional classroom studies.

For this purpose, online proctoring is very much needed. To describe online proctoring, it can be a system where invigilator can guard the examinees online in live times or software where many functions are automated to stop cheating attempts. For huge sum of money and efficiency-related concerns, educational institutions hesitate to use this type of proctoring.

In the above circumstance, we want to conduct research on this online proctoring system, specifically with eye gaze estimation and head pose estimation features. So, the purpose of our research is to find a low-cost solution using computer vision and machine learning to prevent cheating during online examinations so that online education can get the value of normal traditional education.

A. Research Problem

Similar to traditional pen-paper examination, conducting a proper online examination has also become very difficult as the students are involved in various misconduct like, giving group exams or collecting resources from the internet during examination time. For example, at Harvard University, 125

students were caught in their homes for giving a final exam jointly [1].

On the other hand, if the system is fully machine-oriented then it creates an accuracy problem. It is said by an examinee that when she was reading the exam question in loud the proctoring software misjudged her by thinking she was talking with someone else [2]. There are various solutions for increasing the accuracy rate of online proctoring systems. Some of the systems need extra hardware or devices which increases the expense.

So, some of the main problems of online proctoring systems can be mentioned as efficiency and cost. To have accurate results external hardware is used which increases the expense. Full automation or the presence or an invigilator during the whole examination can cause inaccurate detection of cheating attempts. In our research, we tried to address these problems.

II. LITERATURE REVIEW

Bancud and Palconit researched human pose with machine learning to detect cheating [3]. In their research, there is an external device called Jetson Nano used for image capturing and image preprocessing. For pose estimation, they used the OpenPose library and for classifying the pose they used the XGBoost library. A prediction of the cheating can be made from the probability of the classification of the pose. After detecting all the suspicious poses, the frame will be sent to the database for real-time monitoring. In this proposed model a big drawback is that this model needs an external device for processing the videos.

To detect malicious activities during the examination, Hu, Jia, and Fu used head posture and mouth state of examinee's which are collected from a webcam [4]. For head pose estimation, if yaw angle is greater than 15° among 20 frames consecutively, it was declared as an abnormal behavior [4]. Although we have included head pose estimation in our work similar to theirs [4], we also included eye gaze estimation, as a student can face the camera without changing the head pose yet using eye movement can still perform cheating. Also rather than assigning a threshold, we have used hybrid classification where we trained our model to be able to predict cheating attempts.

Similar to our work, in [5] Indi, Pritham, Acharya and Prakasha used eye gaze and head pose to determine a student's attention on the screen. They used a pre-trained FSA-Net model to get three head pose estimates yaw, roll, and pitch. On the condition of whether or not the eye is visible and open, they performed different tasks. They used eye gaze estimation to extract eye gaze estimates for detecting if the eye is invisible or not. Combining these gaze estimates and the head pose angles they pass them as an input to classifier 1-XGBoost. If the eyes are not visible and open, only head pose angles are given to classifier 2 which also works with XGBoost. So, combining dual classifiers, they achieved an accuracy of 96.04%. In contrast, we used a hybrid classifier which includes XGBoost and MLP to predict cheating attempts based on eye gaze estimates and head pose angles.

III. METHODOLOGY

A. System Framework

The entire process in order to create our online proctoring system is shown below using the flowchart.

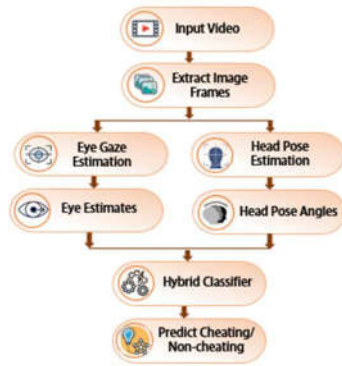


Fig. 1. Proposed model

In our proposed system, we will record students' video through their web camera. From those recorded videos a CSV for each student will be created where all the head and gaze movement's features are recorded with respect to time. After that, for the prediction of cheating attempts, we are using a hybrid classifier. This classifier will flag the time where the student might be involved in cheating. Finally, a report will be given to the exam invigilator with flagged and unflagged times. As the videos are recorded it is a suggestion to the invigilator to visit the flagged time to make sure of the student's activity.

B. Data Collection

Since there is no public dataset on cheating activities of students during examinations, for our research purpose we have created the dataset with the help of ten participants. Each of the participants recorded themselves while performing several online examinations which are available over the internet. There were a total of 30 recorded videos; 2-4 videos per participant. The length of the videos were 4-5 minutes. So the total data is $(4033 \times 18 = \text{row} \times \text{feature})$. The recordings contain both the cheating attempts and non-cheating attempts making them similar to real-life examinations. The videos helped us in finding gaze estimates and head pose angles. We have not made this dataset available for public viewing.

C. Head Pose Estimation

The very first step to implement our proctoring system is estimating the head pose which we were able to do by detecting the head in the extracted images and approximately calculating the head orientation angles. There are three head orientation angles as per Euler's, they are yaw (pan), pitch (tilt), and roll. The pitch is the rotation of the head around the horizontal X-axis and the yaw is the rotation of the head around the vertical Y-axis.

In our system, we have applied only these two horizontal and vertical angles to detect and predict the head position in images by following a pre-trained RealHePoNet model which is a single convolutional neural network model [6].

The head region of the human face must be located in order to determine the head posture angles. To recognize entire human heads, the head detector model built in the study utilizing neural networks [7] is applied. This trained model is built by using SSD (Single Shot Detector) architecture taken from [8]. The neural network architecture used in the model is constructed like this- six convolutional layers (the first layer starts with 32 layers and the next layer will have a double filter from the previous one) and one fully connected hidden layer that contains 512 neurons



Fig. 2. Estimated head poses

D. Eye Gaze Estimation

For the purpose of gaze movement, OpenCV and Dlib (library for machine learning) is used.

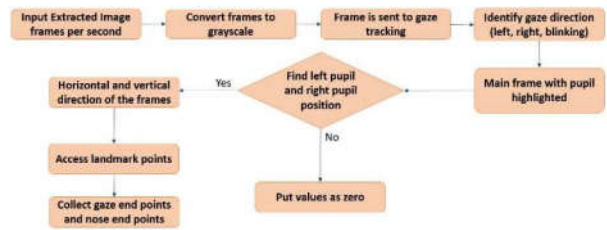


Fig. 3. Flowchart of the proposed eye algorithm

The process of how OpenCV and Dlib work and serves our purpose for finding eye gaze estimates, Fig. 3 is described below in detail.

For OpenCV to work, at first, the images need to be converted to grayscale. Then the advanced 'Canny Edge Detection' algorithm is used in OpenCV to detect the edge of the pupils [9]. The third step is suppression of the non-maximum pixels that means only the pixels which have the max value will be kept. Finally, Hysteresis Thresholding will zero out all the pixels beyond the particular threshold. So everything from the picture will be filtered out except the eye regions. Moreover, the Hough transform is used to detect pupil's locations. From that, we need to collect the pupil's movement. In our eye gaze estimation algorithm, for image processing OpenCV and to detect facial landmarks Dlib is used. Dlib can detect 68 facial landmarks but for our research purpose, we have used only six facial landmarks. They are the extreme corner of the lips, nose, chin, and extreme corner of the eyes. After that those landmarks are sent to the OpenCV function to compute the real-world 3D points with the projected 2D images.

For the calculation of eye gaze estimates, firstly we calculate the eye centers from $leftEyeCenter(x, y)$ and $rightEyeCenter(x, y)$ which are computed with the height and half of the width of the left and right eye landmarks frames which are collected from the Dlib. After that, it is needed to calculate the center of the pupil by finding the centroid $PupilCenterLeft(x, y)$ and $PupilCenterRight(x, y)$ of the image processed and the iris frame. Finally, these will help to calculate the horizontal and vertical ratio of pupil movements.

The equations are:

$$Pupil_Left(x) = \frac{PupilCenterLeft(x)}{(LeftEyeCenter(x) - 2 * 10)} \quad (1)$$

$$Pupil_Right(x) = \frac{PupilCenterRight(x)}{(LeftEyeCenter(x) - 2 * 10)} \quad (2)$$

$$Horizontal_ratio = (Pupil_left(x) + Pupil_Right(x))/2 \quad (3)$$

$$Pupil_Left(y) = \frac{PupilCenterLeft(y)}{(LeftEyeCenter(y) - 2 * 10)} \quad (4)$$

$$Pupil_Right(y) = \frac{PupilCenterRight(y)}{(LeftEyeCenter(y) - 2 * 10)} \quad (5)$$

$$Vertical_ratio = (Pupil_left(y) + Pupil_Right(y))/2 \quad (6)$$

If the horizontal ratio and the vertical ratio are between 0.0-0.5 then the direction is upper and left and if the ratios are between 0.5-1.0 then the direction is lower and right.

IV. EXPERIMENTATIONS

A. Data Pre-processing

The collected data are labeled zero and one very carefully with respect to time. Label '1' indicates that the student is involved in unfair means that is the student is moving his head left or right, moving his eye left or right while facing the monitor and moving both head and eye left or right. So, if

the student's head movement or eye gaze is suspicious at that particular time, then the time is labeled one, otherwise zero where the student is facing the monitor without eye or head movement.

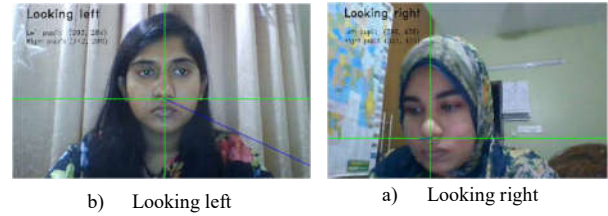


Fig. 4. Estimated eye gaze parameters

The dataset was divided into two parts: training and testing. We used 80% of our dataset for training and 20% for testing in this experiment. Then, we combined the records of all the training videos to train our hybrid model. Also, we made sure to remove all the redundant data from our training CSV so that there is no overfitting in our model. Moreover, since we are taking different types of features, we had to scale all the features. We followed the normalization of the min-max scaling technique and kept all our features within the range zero to one. As a result, our features which are measured in different scales do not create any problem in learning the model.

Along with that, it was seen that some features have a strong correlation with each other. So, for reducing complexity we prefer to remove one of the two features having a correlation greater than 0.9. Then we divided all the properties in the CSV into independent features and dependent variables. Besides 'label', all the properties are independent features as the value of 'label' depends on the value of eye estimates and head pose angles. So 'label' is the dependent variable here.

After preparing this CSV file, this is given to our classifier to predict the outcome.

A hybrid approach by combining two classifications was followed to predict the outcome of our work. Extreme Gradient Boosting otherwise known as XGBoost was used as the first classifier and Multilayer Perceptron (MLP) was used as the second classifier.

Classifier 1: XGBoost: XGBoost is a gradient boosted decision tree implementation in which trees are layered on top of each other to rectify prior trees' errors. So we have our main model and we also have multiple versions of our model and all of them are chained together.

There are some XGBoost hyper parameters like Booster, Eta, Max_depth, etc. whose values eventually determine the learning process of the algorithm. In every step of training, the values are adjusted to achieve maximum accuracy.

The parameters we tuned in Xgboost for better training and testing results are max depth, subsample, n estimators, learning rate. We kept all the other parameters' default values.

Now after we train our XGBoost model for classification, we get the leaf nodes of the tree as predicted values. We used those leaf nodes as an extra parameter and combined them with other features of the dataset when we import the CSV file in our second classifier, MLP.

Classifier 2: MLP: Composed with several perceptrons, a multilayer perceptron is built where data is inserted into the input layers, after completing some processing in hidden layers, the result or prediction against the input is generated from output layers. In our model, we use the activation

function ReLU for the hidden layers as it helps to decrease the vanishing gradient problem and gives a better result. For weight optimization across all the nodes, we tune the parameter solver as 'lbfgs'. Also, tune the max fun for calling the maximum number of the loss functions. Giving an appropriate value of alpha reduces the chance of overfitting and under fitting of our data. We also set the number of epochs by assigning values in max_iter until the convergence. The number of the loss functions is greater than or equal to the max_iter. Moreover, using two hidden layers with the right number of nodes gives the best result for our data. We also set a value for the parameter random state as it decides the number for reproducing the same results. Finally, we get 100% training accuracy and 96.5% accuracy for our testing data.

It is also observed that if the data is only used in XGBoost classifiers then the training accuracy is 100% and the testing accuracy is 91%. Moreover, the data is also trained in a multilayer perceptron neural network with ReLU activation function, and keeping two hidden layers gets training accuracy 100% but the testing accuracy is 87%.

Along with that we train our data in some other commonly used machine learning algorithms. One of the well-known binary classification models is Logistic Regression model which gave a total testing accuracy of 73%. Another effective algorithm is Support Vector Machine (SVM) gave 72% testing accuracy. The Decision Tree Model gave online exam proctoring data an 79% testing accuracy.

Fig. 5, shows the comparison between accuracy of the classifiers, Logistic Regression, Support Vector Machine (SVM), Decision Tree, XGBoost, Multilayer Perceptron (MLP) and our hybrid model.

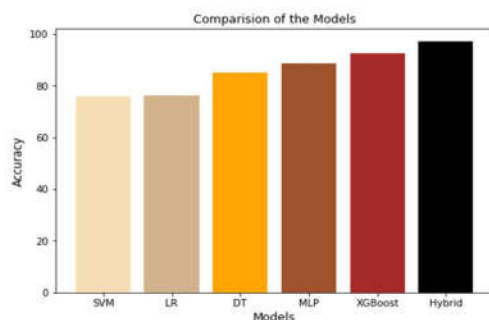


Fig. 5. Comparison of the models

V. RESULT ANALYSIS

The primary objective of this study was to identify a low-cost approach to the most common problem in online examination which is cheating. For our proposed system, we only need a webcam which are already embedded in laptops. So, there is no extra hardware cost for our proctoring system as opposed to [3] where external devices like Jetson Nano were used respectively.

Our hybrid classifier gives an accuracy of 96.5% which is better than using only a single classifier. When used individually, the mentioned classifiers do not provide suitable accuracy. However, the combination of XGBoost and MLP makes our hybrid classifier delivers a very high accuracy which is 96.5%. In online examinations, to predict cheating activities efficiently, this level of accuracy was needed.

In our research, after predicting with hybrid classifiers, we provide a report containing the time period along with the label 0/1 (0 = non-cheating, 1 = cheating) to the invigilator. If a student is flagged for cheating and claims to be wrongly accused, the invigilator can look for that particular time in the recorded video to see whether he is cheating or not.

Though we managed to contribute in this field of study by giving a low cost and efficient approach of combining head pose and eye gaze and finally use of hybrid classifier to predict cheating activities, we also faced some limitations. As there was no readily available dataset for this particular research, we needed to create our own dataset. Given more data, our classification would be more accurate in predicting cheating activities.

REFERENCES

- [1] S. S. Chua, J. B. Bondad, Z. R. Lumapas, and J. D. L. Garcia, "Online examination system with cheating prevention using question bank randomization and tab locking," in *2019 4th International Conference on Information Technology (IncIT)*, 2019, pp. 126–131. doi: 10.1109/INCIT.2019.8912065.
- [2] C. Watkins, A student says test proctoring ai flagged her as cheating whenshe read a question out loud. others say the software could have more dire consequences. <https://sportsgrindentertainment.com>, 4th October 2020.
- [3] G. E. Bancud and E. Palconit, *Human pose estimation using machine learning for cheating detection*, Apr. 2021. doi: 10.13140/RG.2.2.12686.28481.
- [4] S. Hu, X. Jia, and Y. Fu, "Research on abnormal behavior detection of online examination based on image information," in *2018 10th International Conference on Intelligent Human-Machine Systems and Cybernetics (IHMSC)*, vol. 02, 2018, pp. 88–91. doi: 10.1109/IHMSC.2018.10127.
- [5] C. Indi, V. Pritham, V. Acharya, and K. Prakasha, "Detection of malpractice in e-exams by head pose and gaze estimation," *International Journal of Emerging Technologies in Learning (IJET)*, vol. 16, no. 08, pp. 47–60, 2021, issn: 1863-0383. [Online]. Available: <https://online-journals.org/index.php/ijet/article/view/15995>.
- [6] R. Berral-Soler, F. Madrid-Cuevas, R. Muñoz-Salinas, and M. Mañá-Jiménez, *Realheponet: A robust single-stage convnet for head pose estimation in the wild*, Nov. 2020.
- [7] M. J. Marin-Jimenez, V. Kalogeiton, P. Medina-Suarez, and A. Zisserman, *Laeo-net: Revisiting people looking at each other in videos*, 2019. arXiv: 1906.05261 [cs.CV].
- [8] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Ssd: Single shot multibox detector," in *Computer Vision – ECCV 2016*, B. Leibe, J. Matas, N. Sebe, and M. Welling, Eds., Cham: Springer International Publishing, 2016, pp. 21–37, isbn: 978-3-319-46448-0.
- [9] M. Y. Nam, P. K. Rhee, and C. S. Lee, "A real time pupil location system using multiple illuminations clustering under varying environment," in *2009 Digest of Technical Papers International Conference on Consumer Electronics*, 2009, pp. 1–2. doi: 10.1109/ICCE.2009.5012398.